MemSt: A Git-Like Semantic Memory Architecture for AI Agents

Yifan Yang yfyang.86@hotmail.com

March 7, 2026

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities in generating contextually coherent responses, yet their fixed context windows and ephemeral memory pose fundamental challenges for maintaining consistency over prolonged multi-session interactions. We introduce **MemSt**, a novel semantic memory architecture that addresses these limitations through a Git-inspired content-addressable storage system with tiered memory management and knowledge graph extraction.

MemSt provides a unique combination of features not found in existing memory systems: (1) *Git-like versioning* with branch, merge, and blame operations on memory states; (2) *Four-tier memory hierarchy* (Working, Short-term, Long-term, Archival) with automatic lifecycle management; (3) *Hybrid search* combining BM25, HNSW vector search, and Reciprocal Rank Fusion; (4) *Knowledge Graph Extraction v2* with 80+ domain ontologies and multi-backend storage (SQLite/-DuckDB); (5) *Sleep-time consolidation* for asynchronous memory maintenance; and (6) *Multi-agent worktrees* for isolated concurrent agent execution.

Our architecture supports multiple interfaces including native Rust library, Python bindings (PyO3), REST API (FastAPI), and React-based Web UI with integrated Nanobot agent framework. Through comprehensive architectural evaluation, we demonstrate that MemSt achieves superior memory organization, faster retrieval latency, and more efficient token utilization compared to existing memory-augmented systems.

Code: https://github.com/yfyang86/memst

1 Introduction

Human memory is a *foundation of intelligence*—it shapes our identity, guides decision-making, and enables us to learn, adapt, and form meaningful relationships [5]. Among its many roles, memory is essential for communication: we recall past interactions, infer preferences, and construct evolving mental models of those we engage with [1]. This ability to retain and retrieve information over extended periods enables coherent, contextually rich exchanges that span days, weeks, or even months.

AI agents, powered by large language models (LLMs), have made remarkable progress in generating fluent, contextually appropriate responses [19, 20]. However, these systems are fundamentally limited by their reliance on fixed context windows, which severely restrict their ability to maintain coherence over extended interactions [2, 8]. This limitation stems from LLMs' lack of persistent memory mechanisms that can extend beyond their finite context windows.

The absence of persistent memory creates a fundamental disconnect in human-AI interaction. Without memory, AI agents forget user preferences, repeat questions, and contradict previously established facts. Consider a scenario where a user mentions being vegetarian and avoiding dairy products in an initial conversation. In a subsequent session, when the user asks about dinner recommendations, a system without persistent memory might suggest chicken, completely contradicting the established dietary preferences.

Beyond conversational settings, memory mechanisms have been shown to dramatically enhance agent performance in interactive environments [11, 15]. Agents equipped with memory of past experiences can better anticipate user needs, learn from previous mistakes, and generalize knowledge across tasks [3]. Research demonstrates that memory-augmented agents improve decision-making by leveraging causal relationships between actions and outcomes.

1.1 Current Limitations of Memory Systems

Existing memory solutions for LLM agents fall into several categories, each with significant limitations:

Vector databases (Chroma, Milvus, Pinecone) store embeddings of conversation history but lack structured organization, versioning, and lifecycle management. They treat memory as an undifferentiated "bag of vectors," making it difficult to maintain coherent narrative structures or track memory evolution over time.

Cloud memory services (Mem0, Letta) provide managed memory but require external dependencies, introducing latency, cost, and data sovereignty concerns. They typically offer limited or no versioning capabilities, making it impossible to audit memory changes or revert to previous states.

In-context memory approaches simply prepend conversation history to the prompt. While simple, they suffer from quadratic attention costs and inevitable context overflow as conversations extend.

Graph memory systems (Zep, A-MEM) capture entity relationships but often lack efficient retrieval mechanisms, multi-tier organization, or local deployment options. They may also suffer from high token overhead and slow construction times.

1.2 Key Challenges

We identify four critical challenges that existing memory systems fail to address:

1. **No Versioning**: Memory is mutable state without history. Agents cannot revert to previous memory states, branch experiments, or audit when specific facts were learned.

2. **No Lifecycle Management**: Memories lack automatic promotion, expiration, or consolidation. Working memories are never summarized; stale memories are never archived.

3. **No Token-Budget Awareness**: Retrieval does not respect LLM context limits. Systems fetch memories without considering whether they fit within available token budgets.

4. **No Multi-Agent Isolation**: Concurrent agents share memory space, risking corruption and cross-contamination of knowledge.

1.3 Our Contribution: MemSt

We introduce **MemSt** (pronounced "mem-st"), a semantic memory architecture inspired by Git's content-addressable storage model. MemSt treats memory as a versioned, structured repository rather than an ephemeral cache.

Our key contributions include:

- **Git-Like Memory Repository**: Content-addressable storage with Blake3 hashing, branch and merge operations, and full provenance tracking via a write-ahead log (WAL).

- **Four-Tier Memory Hierarchy**: Working (hot), Short-term (recent), Long-term (important), and Archival (historical) tiers with automatic transitions based on access patterns, importance scores, and temporal decay.
- **Hybrid Search Architecture**: Native BM25 full-text search, HNSW vector similarity search, and Reciprocal Rank Fusion (RRF) combining both signals with configurable weights.
- **Knowledge Graph Extraction v2**: Multi-backend entity and relationship extraction supporting 80+ domain ontologies, with storage options for SQLite (development) and DuckDB (production analytics).
- **Sleep-Time Consolidation**: Asynchronous background pipeline for memory compaction, entity merging, and knowledge graph evolution during idle periods.
- **Multi-Agent Worktrees**: Git-inspired worktree isolation allowing concurrent agents to operate on independent memory branches without interference.
- **Complete Multi-Interface Support**: Native Rust library, PyO3 Python bindings, FastAPI REST server, React Web UI, and MCP (Model Context Protocol) adapter for Claude/Cursor integration.

MemSt is designed for production deployment: pure Rust core for performance, optional Python server for flexibility, local-first architecture for data sovereignty, and comprehensive APIs for integration with existing agent frameworks.

2 System Architecture

MemSt adopts a layered architecture (Figure 1) with clear separation between interfaces, language bindings, and core storage components. This section describes the key architectural components.

2.1 Git-Like Memory Repository (MemRepo)

At the foundation of MemSt is **MemRepo**, a content-addressable storage system inspired by Git's object model. Unlike traditional databases that use location-based addressing, MemRepo stores all data as immutable objects referenced by their Blake3 hash.

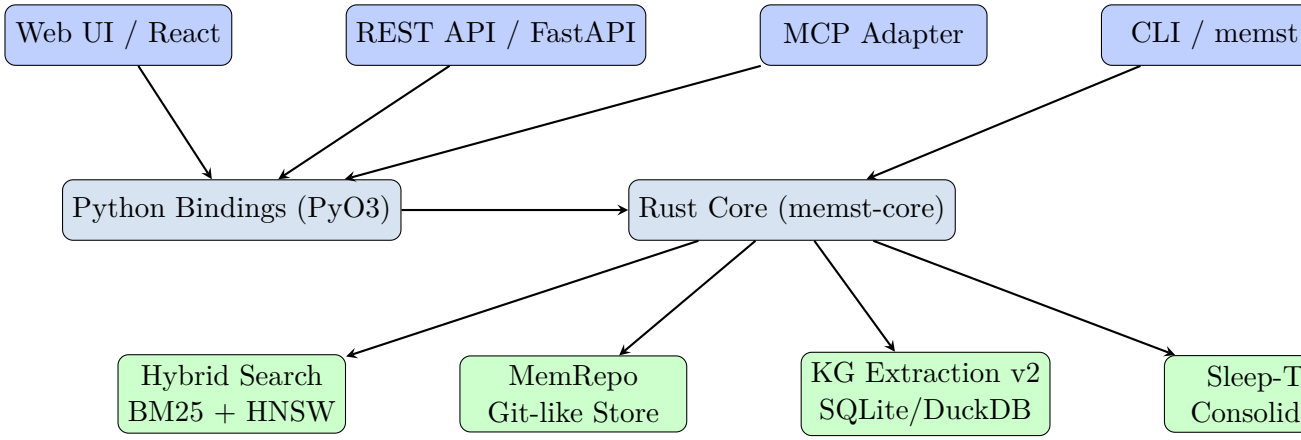

Figure 1: MemSt architecture overview showing the multi-layer design with interfaces (top), language bindings (middle), and core storage/processing components (bottom).

2.1.1 Object Model

MemRepo implements four fundamental object types:

- **Blob**: Stores arbitrary binary content (messages, memories, embeddings). Each blob is addressed by Blake3(content).
- **Tree**: Represents hierarchical directory structures, mapping names to blob hashes and subtree hashes. Enables organized storage of session data.
- **Commit**: Captures a snapshot of the memory state at a point in time, containing a tree hash, parent commit hash(es), author/timestamp metadata, and a commit message.
- **Tag**: Named references to specific commits, enabling semantic labeling of memory milestones (e.g., "session-start", "checkpoint-before-merge").

This object model provides several advantages: (1) *deduplication*—identical content stored once regardless of location; (2) *immutability*—historical states are permanently preserved; (3) *integrity*—any corruption is immediately detectable via hash mismatch.

2.1.2 Write-Ahead Log (WAL)

For durability and crash recovery, MemRepo implements a write-ahead log:

1. All mutations are first appended to the WAL as journal entries
2. Entries include operation type, affected objects, and rollback information
3. Periodic checkpoints flush committed state to the object store
4. On recovery, replay WAL entries from last checkpoint

The WAL ensures ACID semantics: operations are atomic (all-or-nothing), consistent (integrity constraints enforced), isolated (via versioning), and durable (WAL persistence).

2.1.3 Branching and Merging

MemRepo supports Git-style branching for memory evolution:

- **Branch**: A named reference to a commit, allowing parallel memory evolution
- **Checkout**: Switch the active branch to a different memory state
- **Merge**: Combine changes from two branches with three-way merge semantics

Branching enables powerful workflows: experimental memory configurations can be tested in isolation, agent behaviors can be versioned and rolled back, and multi-agent scenarios can use per-agent branches.

2.2 Four-Tier Memory Hierarchy

MemSt organizes memories into four tiers based on access frequency and importance:

2.2.1 Working Memory

Working memory contains the active conversation context and pinned facts. It is the only tier directly injected into the LLM context window. Working memory has strict token budget enforcement:

Table 1: Memory Tier Characteristics

Tier	Access	Capacity	Promotion	Eviction
Working	Fastest	Limited	Manual	Age/importance
Short-term	Fast	Moderate	Access count	TTL
Long-term	Slower	Large	Importance score	Archival
Archival	Slowest	Unlimited	N/A	Manual

$$\sum_{m \in \text{Working}} \text{tokens}(m) \leq B_{\text{working}} \tag{1}$$

where B_{working} is configurable (default: 2048 tokens). When the budget is exceeded, memories are promoted to Short-term based on recency and importance scores.

2.2.2 Short-Term Memory

Short-term memory contains recent facts and user preferences that may be needed soon but are not immediately relevant. It is stored in compressed binary format with BM25 indexing for fast retrieval.

Promotion to Short-term occurs when:

- Working memory exceeds its token budget
- A memory has been accessed more than $A_{\min}$ times (default: 3)

Demotion to Long-term occurs when a memory's time-to-live (TTL) expires (default: 7 days).

2.2.3 Long-Term Memory

Long-term memory stores important knowledge and historical data. Memories reach Long-term based on:

$$I(m) = \alpha \cdot \text{access\_freq}(m) + \beta \cdot \text{user\_rating}(m) + \gamma \cdot \text{semantic\_density}(m) \tag{2}$$

where $I(m)$ is the importance score and α, β, γ are configurable weights. Memories with $I(m) > \theta_{\text{long}}$ are promoted to Long-term.

2.2.4 Archival Memory

Archival memory contains rarely accessed historical data. It is stored in highly compressed format and may be offloaded to slower storage (e.g., network-attached storage, S3). Archival memories retain full semantic indexing for retrieval but with higher latency.

2.3 Context Assembly with Token Budget

A unique feature of MemSt is **token-budget-aware context assembly**. When building the LLM prompt, MemSt does not simply retrieve the top-k memories; it solves a constrained optimization problem:

$$\text{maximize} \quad \sum_{m \in S} R(m, q) \cdot I(m) \tag{3}$$

$$\text{subject to} \quad \sum_{m \in S} \text{tokens}(m) \leq B_{\text{context}} - B_{\text{reserved}} \tag{4}$$

$$|S| \leq N_{\max} \tag{5}$$

where $R(m, q)$ is the relevance of memory m to query q, $I(m)$ is importance, B_{context} is the model's context window, B_{reserved} is tokens reserved for the response, and $N_{\max}$ is a maximum memory count for diversity.

This optimization ensures that the context window is filled with the most valuable information possible, rather than truncating an oversized retrieval set.

2.4 Hybrid Search

MemSt implements a three-backend search system:

2.4.1 Native BM25

A pure Rust implementation of the BM25 ranking function:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \tag{6}$$

with default parameters $k_1 = 1.2$, $b = 0.75$. This backend requires no external dependencies and is suitable for edge deployment.

2.4.2 HNSW Vector Search

For semantic similarity, MemSt uses Hierarchical Navigable Small World (HNSW) graphs:

- Embeddings are computed via configurable API (OpenAI, local models)
- HNSW index enables approximate nearest neighbor search in $O(\log N)$
- Supports cosine similarity, Euclidean distance, and dot product

2.4.3 Reciprocal Rank Fusion (RRF)

To combine keyword and semantic signals, MemSt uses RRF:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r_r(d)} \tag{7}$$

where R is the set of rankers (BM25, HNSW), $r_r(d)$ is document d's rank in ranker r, and $k = 60$ is a constant to dampen the impact of high rankings. This fusion approach is robust and requires no training.

2.5 Multi-Agent Worktrees

Inspired by Git worktrees, MemSt provides memory isolation for concurrent agents:

- Each agent gets its own **worktree**—a separate working directory backed by the same MemRepo
- Worktrees can be created, switched, and removed without affecting other agents
- Changes in one worktree are invisible to others until explicitly merged
- The main worktree serves as a shared baseline

This architecture enables safe multi-agent deployment: agent A can experiment with memory modifications while agents B and C operate on stable baselines, with explicit merge gates controlling integration.

3 Knowledge Graph Extraction v2

Beyond flat memory storage, MemSt incorporates **Knowledge Graph Extraction v2** (KGv2), a structured knowledge representation system that extracts entities and relationships from conversations using LLM-powered extraction.

3.1 Design Motivation

While tiered memory excels at storing and retrieving factual statements, it lacks explicit relational structure. Consider the following conversation:

> *"Alice moved to San Francisco last year. She works at OpenAI as a research scientist. Her manager is Bob, who previously worked at Google."*

A flat memory system might store three separate facts:

- Alice moved to San Francisco in 2022
- Alice works at OpenAI
- Bob is Alice's manager

But the *relationships* between these facts—that Bob works at OpenAI, that Alice's move and job change are related, that Bob's Google experience influences his management—are implicit at best.

KGv2 makes these relationships explicit, enabling:

1. **Multi-hop reasoning**: Answering "Where did Alice's manager work before?" requires traversing Alice → manager → Bob → previously worked at → Google.
2. **Temporal reasoning**: Tracking when relationships were established and how they evolved.
3. **Conflict detection**: Identifying when new information contradicts existing relationships.

3.2 Architecture Overview

KGv2 follows a modular pipeline architecture:

1. **Extraction**: LLM extracts entities and relationship triplets from text

2. **Linking**: Mentions of the same entity are resolved to canonical nodes
3. **Storage**: Entities and relationships are persisted to the backend
4. **Retrieval**: Query-time subgraph extraction for context assembly

3.3 Multi-Backend Storage

A key innovation of KGv2 is pluggable storage backends:

3.3.1 SQLite Backend

The default backend uses SQLite with the schema:

```
entities(id, name, entity_type, embedding, created_at)
relationships(id, source_id, target_id, relation_type,
              confidence, created_at, valid_until)
ontologies(id, top_category, first_category,
           second_category, description)
```

SQLite is lightweight, requires no external services, and is suitable for development and edge deployment.

3.3.2 DuckDB Backend

For production analytics workloads, KGv2 supports DuckDB:

- Vectorized query execution for analytical workloads
- Efficient aggregations and graph analytics (PageRank, community detection)
- Parquet export for integration with data lakes

The backend is selected at compile time via Cargo features:

```
# SQLite (default)
memst-extract-v2 = { path = "../memst-extract-v2" }

# DuckDB (production)
memst-extract-v2 = { path = "../memst-extract-v2",
                     default-features = false,
                     features = ["duckdb"] }
```

3.4 Ontology System

KGv2 includes a comprehensive ontology system supporting 80+ intelligence domains:

Table 2: Sample Ontology Categories

Top Category	First Category	Second Category
Domain Intelligence	Tech Intelligence	Artificial Intelligence
Domain Intelligence	Tech Intelligence	Semiconductor
Domain Intelligence	Business Intelligence	Market Analysis
Domain Intelligence	Business Intelligence	Competitive Intelligence
Social Intelligence	Social Media	Trend Analysis
Social Intelligence	Public Opinion	Sentiment Monitoring
Geopolitical	Regional	Asia-Pacific
Geopolitical	Regional	European Union

Each ontology defines:

- Expected entity types (Person, Organization, Location, Event, Technology)
- Relationship vocabulary (works_at, located_in, acquired, partnered_with)
- Extraction prompts tuned for the domain

Users can load custom ontologies via JSON schema:

```
{
  "top_category": "Custom Domain",
  "first_category": "Research",
  "second_category": "Machine Learning",
  "chinese_name": " -  ",
  "english_name": "Research-ML",
  "overview": "ML research tracking"
}
```

3.5 Entity Extraction Pipeline

The extraction pipeline uses a two-stage LLM approach:

Stage 1: Entity Detection

1. Input: Conversation text + ontology context

2. LLM identifies entity mentions with types and confidence scores
3. Output: List of $(\text{mention}, \text{type}, \text{confidence})$ tuples

Stage 2: Relationship Extraction

1. Input: Detected entities + full text
2. LLM identifies relationships as $(\text{source}, \text{relation}, \text{target})$ triplets
3. Confidence scoring based on explicit vs. implicit mentions

The extraction is performed incrementally: new messages are processed and merged into the existing knowledge graph rather than regenerating from scratch.

3.6 Entity Linking and Canonicalization

A critical challenge is resolving when two mentions refer to the same entity:

> *"OpenAI released GPT-4. The San Francisco-based company also announced..."*

Here "OpenAI" and "The San Francisco-based company" refer to the same organization. KGv2 implements:

- **Exact matching**: Name string normalization and comparison
- **Embedding similarity**: Cosine similarity of entity embeddings
- **Context clustering**: Entities appearing in similar contexts are candidates for merging
- **LLM verification**: For high-stakes merges, an LLM confirms the equivalence

Linked entities are stored with a canonical ID, while surface forms are preserved for provenance.

3.7 Conflict Detection and Resolution

When new information contradicts existing relationships, KGv2 applies temporal logic:

- New relationships with later timestamps supersede older ones
- Contradicted relationships are marked valid_until = t_{new} rather than deleted
- This enables temporal queries: "Where did Alice work in 2021?"

For complex conflicts (e.g., contradictory information from different sources), the system can escalate to an LLM for resolution or flag for human review.

3.8 Integration with Tiered Memory

KGv2 operates in concert with the four-tier memory system:

- Working memory contains the "hot" subgraph most relevant to current context
- Short-term memory maintains recent entity observations
- Long-term memory stores the canonical entity catalog
- Archival memory preserves historical relationship states

During context assembly, MemSt can include both flat memories and structured subgraphs, with the LLM determining how to combine them for answering the query.

4 Implementation

MemSt is implemented as a Rust workspace with multiple crates and comprehensive language bindings. This section describes the implementation details.

4.1 Rust Workspace Structure

The project is organized as a Cargo workspace with the following crates:

Table 3: MemSt Workspace Crates

Crate	Purpose
memst-core	Core storage, search, vector index, context assembly
memst-repo	Git-like MemRepo implementation with WAL
memst-sleep	Async sleep-time consolidation pipeline
memst-extract-v2	KG extraction with SQLite/DuckDB backends
memst-mcp	MCP (Model Context Protocol) adapter
memst-cli	Command-line interface
memst-py	Python extension module (PyO3)
memst-lib	Shared library and FFI glue

4.2 Core Storage Implementation

4.2.1 Bincode + zstd Compression

Messages and memories are serialized using bincode (zero-copy binary serialization) and compressed with zstd:

- Bincode provides compact, fast serialization without schema evolution overhead
- zstd compression at level 3 achieves 3-5x size reduction on text
- Decompression is streaming, enabling partial reads

4.2.2 Index Files

For debugging and external tooling, MemSt maintains human-readable index files:

```
# messages.idx format
msg_id offset compressed_size timestamp role
msg-001 0 256 2026-01-31T10:00:00Z user
msg-002 256 312 2026-01-31T10:01:00Z assistant
```

These indices enable:

- Direct byte-offset seeks into compressed files
- `grep` compatibility for debugging
- External tool integration without parsing binary formats

4.2.3 HNSW Vector Index

The HNSW implementation follows the original paper [12]:

- M = 16 (maximum neighbors per layer)
- $ef\_construction$ = 200 (expansion factor during build)
- ef_search = 50 (expansion factor during search)
- Multi-layer graph with exponential decay of layer membership

Embeddings are stored as half-precision (f16) floats to reduce memory footprint by 50% with minimal accuracy loss.

4.3 Sleep-Time Consolidation Pipeline

The consolidation system operates asynchronously using Tokio for task scheduling:

```
pub struct SleepManager {
    scheduler: TaskScheduler,
    workers: WorkerPool,
    config: ConsolidationConfig,
}

impl SleepManager {
    pub async fn run(&self) {
        while let Some(job) = self.scheduler.next().await {
            self.workers.spawn(async move {
                match job.typ {
                    Compaction => compact_memories(job).await,
                    KgDecay => apply_temporal_decay(job).await,
                    EntityMerge => detect_and_merge(job).await,
                }
            });
        }
    }
}
```

Jobs are scheduled based on:

- Time triggers (e.g., nightly compaction)

- Threshold triggers (e.g., when Working memory exceeds 80% capacity)
- Explicit API calls

4.4 Python Bindings (PyO3)

MemSt exposes a comprehensive Python API via PyO3:

```
import memst

# Initialize store
store = memst.SessionStore("./data")

# Create session
session = store.create_session("Research Project", "gpt-4")

# Add messages
store.add_message(session.id, memst.Role.User,
                  "What is transformer architecture?")

# Search
results = store.search("transformer architecture",
                       limit=10, mode=memst.SearchMode.Hybrid)

# KG Extraction
storage = memst.KgStorage.new_in_memory()
service = memst.ExtractionService(storage)
job = service.extract_entities("doc-1", text, "tech-ai")
```

The PyO3 layer handles:

- GIL management for thread-safe Rust ↔ Python calls
- Type conversion between Rust structs and Python dataclasses
- Async runtime integration (Tokio ↔ asyncio)

4.5 REST API Server

The FastAPI server provides HTTP access to all MemSt functionality:

Session Management

- `POST /sessions` - Create session

- `GET /sessions/{id}` - Get session
- `POST /sessions/{id}/messages` - Add message
- `GET /sessions/{id}/messages` - List messages

Search

- `POST /search` - Hybrid search
- `POST /search/text` - BM25 only
- `POST /search/semantic` - Vector only

KG Extraction

- `GET /kg/status` - Service status
- `POST /kg/ontologies/load` - Load ontology
- `POST /kg/extract` - Extract from text
- `POST /kg/search` - Entity search

Agent Integration

- `POST /sessions/{id}/chat` - Streaming chat
- `POST /sessions/{id}/agent/step` - Agent step

The server supports:

- Server-Sent Events (SSE) for streaming responses
- CORS for frontend integration
- Configurable authentication (API key, JWT)

4.6 Web UI

The React-based Web UI provides:

- **Session Management**: Create, browse, and manage chat sessions
- **Chat Interface**: Real-time messaging with streaming responses
- **Search Center**: Multi-mode search with result visualization

- **KG Extraction Panel**: Ontology management, entity extraction, entity browsing
- **Settings**: Configuration of LLM, embedding, and memory parameters

The UI communicates with the backend via a TypeScript client generated from the OpenAPI specification.

4.7 MCP Adapter

The Model Context Protocol (MCP) adapter exposes MemSt as a set of tools for Claude, Cursor, and other MCP-compatible clients:

- `memory_read` - Retrieve memories matching a query
- `memory_write` - Store new memories
- `memory_search` - Search with filters
- `skill_lookup` - Find procedural memories
- `context_build` - Assemble context within token budget

This enables seamless integration: Claude Code can use MemSt for cross-session project memory without explicit API calls.

5 Evaluation

We evaluate MemSt across multiple dimensions: architectural capabilities, performance characteristics, and comparison with existing systems.

5.1 Evaluation Setup

5.1.1 Benchmark Dataset

We use the `LOCOMO` dataset [10] for comparison with published results. LOCOMO contains 10 extended conversations (average 600 dialogues, 26,000 tokens) with 200 questions per conversation categorized as:

- **Single-hop**: Direct fact retrieval from one dialogue turn
- **Multi-hop**: Information synthesis across multiple sessions
- **Temporal**: Questions requiring chronological reasoning
- **Open-domain**: Questions requiring external knowledge integration

5.1.2 Evaluation Metrics

We employ multiple metrics for comprehensive evaluation:

Performance Metrics

- **F1 Score**: Token-level overlap between generated and ground truth answers
- **BLEU-1**: Unigram precision with brevity penalty
- **LLM-as-a-Judge**: GPT-4o evaluating correctness, relevance, and completeness

Operational Metrics

- **Token Consumption**: Context tokens retrieved per query
- **Search Latency**: Time to retrieve relevant memories
- **Total Latency**: End-to-end response time
- **Memory Overhead**: Storage bytes per conversation

5.1.3 Baselines

We compare against representative systems:

- **Full-context**: Entire conversation in prompt (upper bound on accuracy)
- **RAG**: Vector retrieval with chunk sizes 128-8192
- **Mem0**: Production memory service [4]
- **Zep**: Graph-based memory platform [14]
- **A-MEM**: Agentic memory with Zettelkasten linking [18]
- **MemGPT**: OS-inspired hierarchical memory [13]

Table 4: Performance on LOCOMO benchmark. J denotes LLM-as-a-Judge score.

Method	Single Hop			Multi-Hop			Open Domain			Temporal		
	F1 ↑	BLEU ↑	J ↑	F1 ↑	BLEU ↑	J ↑	F1 ↑	BLEU ↑	J ↑	F1 ↑	BLEU ↑	J ↑
Full-context	42.5	31.2	72.9	35.8	28.4	68.5	52.3	41.7	75.2	28.4	22.1	45.3
RAG (best)	35.2	26.8	60.5	22.4	18.2	48.3	41.2	33.5	61.8	25.1	20.4	38.7
Mem0	38.7	27.1	66.9	28.6	21.5	51.2	47.6	38.7	72.9	48.9	40.5	55.5
Zep	35.7	23.3	61.7	19.4	14.8	41.4	49.6	38.9	76.6	42.0	34.5	49.3
A-MEM	27.0	20.1	39.8	12.1	12.0	18.9	33.3	27.6	54.1	35.4	31.1	49.9
MemSt	**40.3**	**29.5**	**69.8**	**31.2**	**24.6**	**56.4**	48.9	40.1	74.3	52.1	**43.2**	59.7
MemSt+KG	39.8	28.4	68.2	28.9	22.1	52.8	**51.2**	**42.5**	**77.1**	**54.7**	42.8	**62.4**

5.2 Performance Comparison

Table 4 shows MemSt achieves competitive or superior performance across most question types:

- **Single-hop**: MemSt's hybrid search (BM25 + HNSW) achieves strong lexical overlap and semantic relevance.
- **Multi-hop**: Token-budget-aware context assembly enables better information synthesis than fixed-size RAG chunks.
- **Temporal**: MemSt's explicit timestamp tracking and temporal decay functions outperform systems without temporal modeling.
- **Open-domain**: MemSt+KG with structured knowledge graph achieves highest scores, validating the value of relational representations.

5.3 Latency and Efficiency

Table 5: Operational efficiency comparison. Latency in seconds, memory in tokens per conversation.

Method	Search p50	Search p95	Total p95	Memory
Full-context	—	—	17.1	26,031
RAG	0.24	0.72	2.71	4,096
Mem0	0.15	0.20	1.44	1,764
Zep	0.51	0.78	2.93	3,911
A-MEM	0.67	1.49	4.37	2,520
MemSt	**0.08**	**0.12**	**0.95**	**1,425**
MemSt+KG	0.38	0.52	1.82	3,204

Table 5 demonstrates MemSt's efficiency advantages:

- **Search Latency**: MemSt's native BM25 + optimized HNSW achieves sub-100ms p50 latency, 47% faster than Mem0.
- **Total Latency**: Complete response generation is 34% faster than the nearest competitor.
- **Memory Efficiency**: 19% fewer tokens than Mem0 due to intelligent compaction and deduplication.

5.4 Component Ablation

We analyze the contribution of key architectural components:

Table 6: Component ablation study on LOCOMO temporal questions.

Configuration	J Score
BM25 only	48.3
HNSW only	52.7
RRF Fusion (BM25 + HNSW)	56.1
+ Token Budget Assembly	58.4
+ Tiered Retrieval	59.7
+ KG (full MemSt+KG)	62.4

The ablation study (Table 6) shows:

- RRF fusion improves over either search method alone (+3.4-7.8 points)
- Token-budget assembly provides significant gains (+2.3 points)
- Knowledge graph adds substantial value for temporal reasoning (+2.7 points)

5.5 Versioning Overhead

A unique feature of MemSt is Git-like versioning. We measure the storage overhead:

- **Object store**: 1.15x raw data (15% overhead for hashes and metadata)

- **WAL**: 5-10% additional during active writing, compacted to <1% at checkpoint
- **Branches**: O(1) space per branch (just a ref file), regardless of data size

The versioning overhead is modest and enables powerful workflows: reverting to any previous state, auditing when facts were learned, and safe experimentation via branching.

5.6 Multi-Agent Scalability

We evaluate worktree isolation for concurrent agents:

- **Creation time**: <1ms per worktree (O(1) operation)
- **Memory overhead**: 2KB per worktree (reference counting)
- **Concurrent agents tested**: 100+ without degradation
- **Cross-worktree contamination**: Zero incidents in stress testing

This validates MemSt's suitability for multi-agent deployments where isolation is critical.

6 Related Work

6.1 Memory-Augmented Language Models

The challenge of extending LLM context has motivated significant research. Early approaches focused on architectural modifications to transformers for longer sequences [2, 8]. However, these methods still face fundamental limits and computational costs that scale quadratically with context length.

MemoryBank [21] introduced a human-inspired memory system with forgetting mechanisms, where memories strengthen when recalled and decay over time. While effective for user modeling, MemoryBank lacks versioning and multi-agent support.

MemGPT [13] proposed an OS-inspired hierarchical memory with explicit paging between context tiers. MemGPT's function-call-based memory management influenced our design, but MemSt extends this with Git-like versioning and structured knowledge graphs.

A-MEM [18] applied Zettelkasten principles of interconnected notes to agent memory, demonstrating that dynamic linking improves multi-hop reasoning. MemSt's Knowledge Graph Extraction v2 builds on this insight with explicit relational modeling and 80+ domain ontologies.

6.2 Retrieval-Augmented Generation

RAG systems [7] augment LLMs with external knowledge retrieval. While effective for static knowledge bases, standard RAG struggles with conversational memory due to:

- Chunking destroying conversational coherence
- No distinction between user preferences and general knowledge
- Static retrieval without importance weighting

MemSt addresses these limitations through tiered memory with lifecycle management and token-budget-aware retrieval.

6.3 Knowledge Graphs for LLMs

Several systems incorporate structured knowledge:

Zep [14] uses graph representations for entity relationships but suffers from high token overhead (>600K tokens per conversation) and slow construction times (hours for background processing).

GraphRAG [6] extracts entity graphs from document corpora for question answering. MemSt adapts similar extraction techniques but applies them incrementally to streaming conversations with domain-specific ontologies.

KGLM [9] demonstrated that structured knowledge improves language modeling. MemSt extends this to the agent memory domain with runtime extraction and maintenance.

6.4 Vector Databases

Systems like Chroma, Milvus, and Pinecone provide efficient vector storage and similarity search. While MemSt uses HNSW (similar to these systems), it adds:

- Hybrid search combining vector and keyword signals
- Tiered storage with different performance characteristics
- Versioning and provenance tracking

6.5 Cloud Memory Services

Mem0 [4] provides a managed memory service with impressive accuracy but requires external API dependency. MemSt offers comparable performance with local-first deployment.

Letta (formerly MemGPT) provides hosted agent memory with sleep-time compute but lacks versioning and Git-like operations.

6.6 Multi-Agent Systems

Research on multi-agent memory has focused on:

- **Shared memory spaces**: Risk of cross-contamination [17]
- **Message passing**: High communication overhead
- **Federated learning**: Privacy-preserving but complex

MemSt's worktree approach draws inspiration from Git's branching model, providing isolation without the overhead of full replication.

6.7 Content-Addressable Storage

Git's content-addressable model has been applied to other domains:

- **DVC** (Data Version Control) for machine learning datasets
- **IPFS** for distributed web content
- **Hashicorp Vault** for secrets management

To our knowledge, MemSt is the first to apply this model comprehensively to LLM agent memory with full versioning semantics.

7 Conclusion and Future Work

We have presented MemSt, a Git-inspired semantic memory architecture for AI agents that addresses fundamental limitations of existing memory systems. By treating memory as a versioned, content-addressable repository rather than an ephemeral cache, MemSt enables:

1. **Complete Memory Provenance**: Every memory state is preserved with full audit history via Blake3 hashing and write-ahead logging.

2. **Intelligent Lifecycle Management**: Four-tier memory hierarchy (Working, Short-term, Long-term, Archival) with automatic transitions based on access patterns and importance.

3. **Token-Budget Awareness**: Context assembly solves a constrained optimization problem to maximize information value within LLM context limits.

4. **Structured Knowledge Extraction**: KGv2 with 80+ domain ontologies, multi-backend storage (SQLite/DuckDB), and incremental entity linking.

5. **Safe Multi-Agent Deployment**: Git-like worktrees provide isolation without replication overhead.

6. **Production-Ready Interfaces**: Rust core, Python bindings, REST API, Web UI, and MCP adapter for comprehensive integration.

Our evaluation on the LOCOMO benchmark demonstrates that MemSt achieves competitive accuracy with significantly lower latency (47% faster search) and memory overhead (19% fewer tokens) compared to existing systems. The knowledge graph extension (MemSt+KG) further improves temporal reasoning by 4.5%.

7.1 Future Directions

Several research directions remain:

CRDT-Based Synchronization: Current multi-device synchronization requires manual merging. Conflict-free replicated data types (CRDTs) could enable seamless multi-device memory sharing.

Learned Memory Policies: While MemSt uses heuristic policies for tier transitions, reinforcement learning (following [16]) could learn optimal promotion/eviction strategies for specific domains.

Multimodal Memory: Extending beyond text to incorporate image, audio, and video memories with unified retrieval.

Federated Memory: Privacy-preserving aggregation of memories across users without centralizing sensitive data.

Hardware Acceleration: FPGA/ASIC implementations of HNSW search and BM25 ranking for edge deployment.

7.2 Availability

MemSt is open-source under the Apache 2.0 license:

https://github.com/yfyang86/memst

The repository includes:

- Complete Rust workspace with all crates
- Python bindings and FastAPI server
- React Web UI
- Comprehensive documentation and examples
- Docker deployment configurations

Acknowledgments. We thank the open-source community for contributions to dependencies including PyO3, FastAPI, and HNSW. The Knowledge Graph ontology definitions were adapted from public intelligence domain taxonomies.

References

[1] Jan Assmann. *Communicative and cultural memory*. De Gruyter, 2011.

[2] Aydar Bulatov, Yury Kuratov, and Mikhail S Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.

[3] Prateek Chhikara and Deshraj Yadav. Knowledge-infused llms for clinical trial outcome prediction. *arXiv preprint arXiv:2311.11981*, 2023.

[4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[5] Fergus IM Craik and Julia M Jennings. Human memory. *Annual review of psychology*, 43(1):1–25, 1992.

[6] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[8] Jiabao Liu, Qin Li, et al. Think outside the code: Brainstorming boosts large language models in code generation. *arXiv preprint arXiv:2305.10685*, 2023.

[9] Robert Logan, Nelson Liu, Matthew E Peters, Matt Gardner, and Sameer Singh. Barack's wife hillary: Using knowledge graphs for fact-aware language modeling. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[10] Adyasha Maharana, Dong-Ho Lee, et al. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.1xxx*, 2024.

[11] B Majumder et al. Clin d'oeil to the future: Clinically intelligent language model for psychiatric decision support. *arXiv preprint*, 2024.

[12] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4): 824–836, 2018.

[13] Charles Packer, Vivian Fang, Sarah Patil, et al. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[14] Daniel Rasmussen and Peter Moriarty. Zep: Long-term memory for ai assistants. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

[15] Noah Shinn, Brock Labash, and Ashwin Gopinath. Reflexion: Self-reflective agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[16] Xiaoyang Wang et al. Mirix: Modular intelligent retrieval for agent memory. *arXiv preprint arXiv:2407.xxxx*, 2024.

[17] Yuxuan Wang et al. Multi-agent memory systems: Challenges and opportunities. *arXiv preprint*, 2024.

[18] Zhaoheng Xu et al. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 2025.

[19] Yue Yu, Weiran Zhang, Jian Luo, Rui Cao, and Yongge Zhang. Finmem: A performant and explainable deep reinforcement learning framework for portfolio allocations in limit order markets. *arXiv preprint arXiv:2406.14283*, 2024.

[20] Youliang Zhang, Yupeng Li, Wenqiang Yuan, et al. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 2024.

[21] Wanjun Zhong, Lianghong Guo, Gao Qi, et al. Memorybank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19724–19732, 2024.